Programação Concorrente

Introdução aos Sistemas Operacionais - Parte 04

Professor: **Jeremias Moreira Gomes**
E-mail: jeremias.gomes@idp.edu.br

Estruturas de Sistemas Operacionais

Introdução

- Até agora, foi visto como o Sistema Operacional “parece” por fora
  - Interface a qual o programador tem acesso
- Essa aula tem por objetivo olhar por dentro do SO
  - Estrutura
  - Finalidade
  - Possibilidades
  - Vantagens e Desvantagens

Sistemas Monolíticos

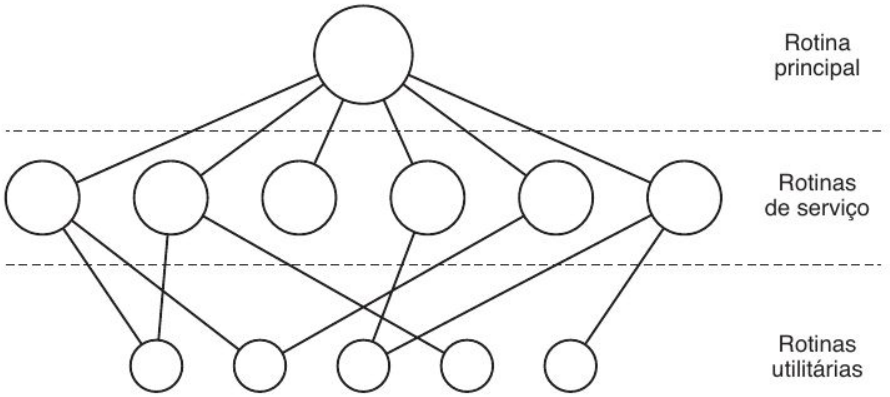
- Organização mais comum
- O SO é executado como um **único programa em modo núcleo**
- SO é escrito como rotinas e ligado a um único binário
  - Coleção de rotinas para cada uma das suas atribuições
- Cada procedimento pode chamar qualquer procedimento
  - Extremamente eficiente
  - Porém é um caos ter esse grau de liberdade

Sistemas Monolíticos

- Maior **desvantagem** é que **uma quebra do sistema derruba todo o SO**
- Não há ocultação de informações (a nível de sistema)
    - Toda rotina enxerga toda outra outra rotina e vice-versa
- É possível ter uma estrutura melhor, separando as chamadas por rotinas e utilizando locais definidos para utilização de parâmetros das chamadas (pilha, por exemplo)
    - Dessa forma, uma instrução é executada por desvio de controle (*trap*)

Sistemas Monolíticos

- Utilizando essa organização, temos a seguinte estrutura para o SO:
  a. Um programa invoca uma rotina de serviço
  b. Um conjunto de rotinas executam chamadas de sistema
  c. Rotinas utilitárias auxiliam rotinas de serviço.



Sistemas Monolíticos

- Cada chamada de sistema tem uma rotina de serviço encarregada da mesma
- Há também suporte a extensões carregáveis sob demanda
  a. Drivers de dispositivos de E/S
  b. Sistemas de Arquivos
- Essas extensões são conhecidas como:
  a. Bibliotecas Compartilhadas
  b. *Dynamic Link Libraries* (DLLs)

Sistemas de Camadas

- **Generalização da estrutura apresentada no sistema monolítico**
- Desenvolvido por E. W. Dijkstra em conjunto com seus estudantes
  - Sistema chamado de THE (Electrologica X8 (32K e palavras de 27 bits))
- O sistema possui 6 camadas.

Camada	Função
5	O operador
4	Programas de usuário
3	Gerenciamento de entrada/saída
2	Comunicação operador–processo
1	Memória e gerenciamento de tambor
0	Alocação do processador e multiprogramação

Sistemas de Camadas

A. **Camada 0**

   a. **Alocação de Processos**

      i. Chaveamento em interrupções

      ii. Expiração de temporizadores

      iii. Tudo acima da camada 0 é visto de forma sequencial

   b. Multiprogramação básica da CPU

Sistemas de Camadas

B. Camada 1

a. Gerenciamento de Memória.

i. Alocar espaço para processos na memória principal

1. Também um tambor magnético de 512K (no THE)

ii. Gerenciar partes de processos sem espaço na memória principal

iii. Acima da camada 1 não é necessário preocupar-se com localidade

Sistemas de Camadas

C. **Camada 2**

   a. **Comunicação entre Processo e Console de Operação**

      i. Console de operação é o usuário

      ii. Cada processo tem seu console de operação

D. **Camada 3**

   a. **Gerenciamento de Dispositivos de Entrada e Saída**

      i. Armazenamento temporário de fluxos de informação

Sistemas de Camadas

E. **Camada 4**

   a. **Programas de Usuários**

      i. Uso sem preocupação com processos, memória, console ou Entrada e Saída

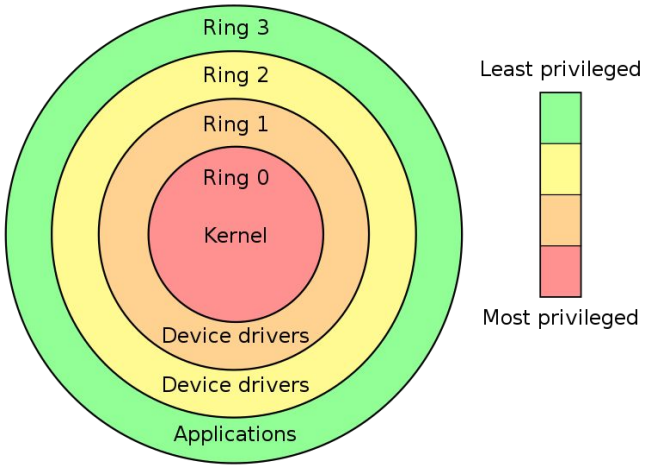
F. **Camada 5**

   a. **Processo Operador**

      i. Localização do operador do sistema

Sistemas de Camadas

- Outro sistema que utilizou a estrutura em camadas foi o MULTICS
  - Chamou a abordagem de anéis concêntricos.
  - Quanto mais próximo do centro, mais privilegiado.



Sistemas de Camadas

- MULTICS
  - Quando um procedimento quer chamar um procedimento em um anel inferior ao seu, ele realiza uma *trap*
  - Parâmetros eram meticulosamente checados antes de prosseguir
  - Dessa forma, tornou-se possível rotinas individuais protegidas
    - Leitura, escrita ou gravação

Sistemas de Camadas

- MULTICS
  - O sistema em anéis **permitia estruturar subsistemas de usuário**.
    - Professor separar acesso de seus alunos
- No THE, a implementação era similar, mas apenas como forma de ver o sistema funcionando
  - Só no MULTICS as validações de segurança funcionavam

Microkernel

- A partir do sistema em camadas, projetistas têm a opção de traçar limites
- Todas as camadas podem acessar o núcleo, mas não é necessário
    - Se está no núcleo ou com acesso no núcleo, um erro pode derrubar o sistema
- Já processos de usuário, com menos poder, podem não ser fatais
- Sistemas Operacionais são tão sujeitos a erros que fabricantes já colocam botão de reinicialização

Microkernel

- A ideia básica do microkernel (ou micronúcleo) dividir o sistema em pequenos módulos bem definidos
  - Somente o microkernel executa em modo núcleo
  - Outros módulos executam em modo usuário
  - Melhorar a confiabilidade de funcionamento
- Um driver de dispositivo como processo de usuário que para de funcionar, não compromete o sistema

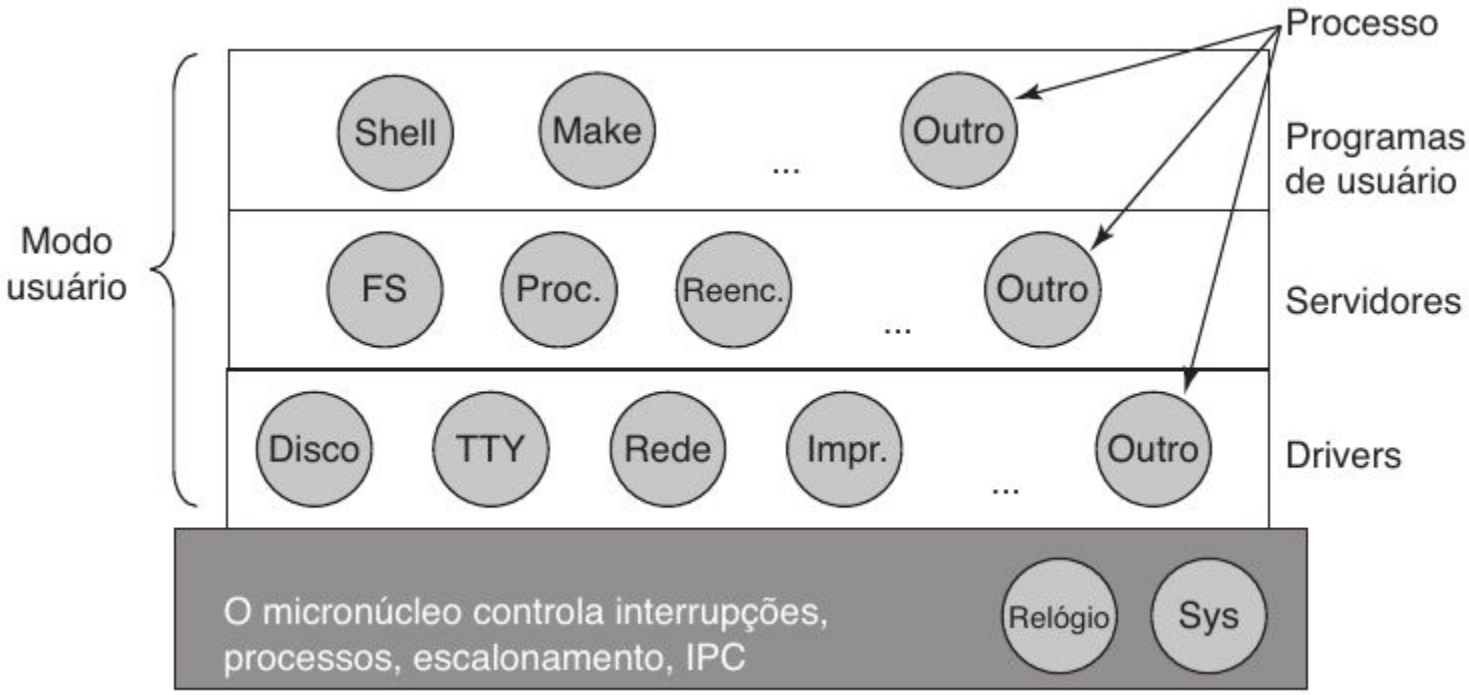
Microkernel

- A maioria dos desktops atuais não utilizam microkernel
- Microkernel aparece mais em sistemas de tempo real, industriais e militares
- Micronúcleos famosos:
  - Symbian e MINIX 3

Microkernel - MINIX 3

- Levou a ideia de modularidade ao extremo.
- Pouco mais de 12 mil linhas de código.
  - 12.000 linhas de C
    - Escalonamento de processos, comunicação e chamadas de núcleo.
  - 1.400 linhas de assembler
    - Capturar interrupções e chavear processos

Microkernel - MINIX 3



Microkernel - MINIX 3

- **No núcleo**:
  a. Associar tratadores de interrupção
  b. Transferir dados
  c. Instalar mapas de memórias para novos processos
- **Fora do núcleo**:
  a. Drivers de dispositivos
  b. Servidores
  c. Programas de usuário

Microkernel - MINIX 3

A. Drivers de dispositivos:
   - Executado em modo usuário
   - Não tem acesso físico ao espaço de portas de E/S
   - Solicita escritas ao núcleo
     i. Consegue checar autorizações de dispositivos.
     ii. Áudio não vai conseguir acessar disco, por exemplo

Microkernel - MINIX 3

B. Servidores:

- Maior parte do trabalho do SO
  i. Servidores de arquivos
  ii. Gerenciamento de processos
  iii. Etc
- Programas de usuários fazem requisições por meio de mensagens
- Há um serviço somente para conferir se outros serviços estão funcionando
  i. Sistema regenera a si mesmo

Microkernel - MINIX 3

- Cada driver e servidor tem poder para fazer só o seu propósito
- A ideia do microkernel está relacionada a separar minuciosamente mecanismo e política
  - Exemplo - Escalonamento de Processos
    - Um algoritmo de escalonamento pode gerar as prioridades em modo usuário (política)
    - O sistema, de posse das prioridades irá procurar o processo e executá-lo (mecanismo)

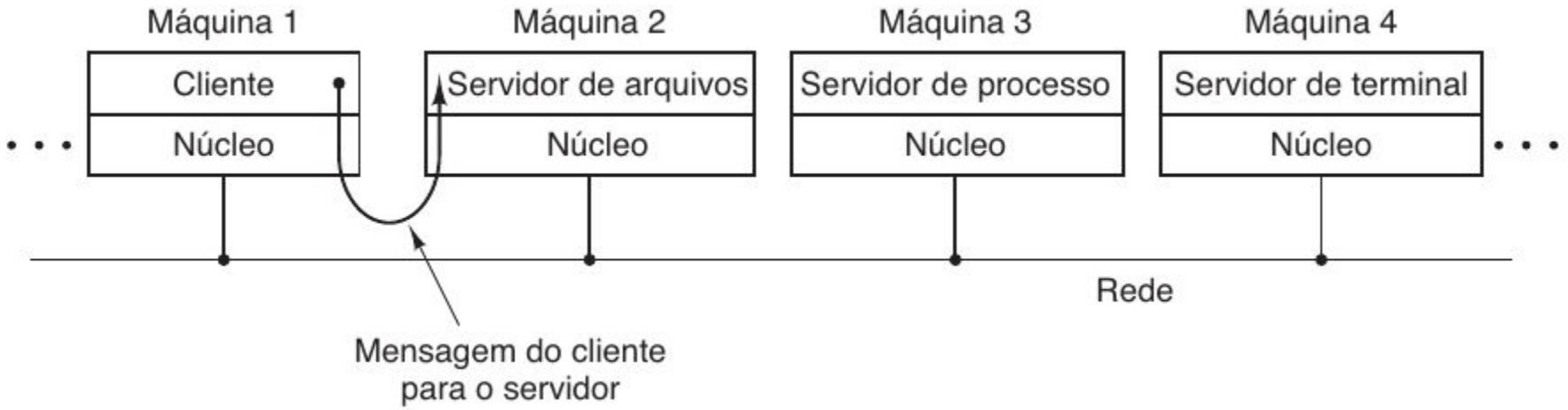
Modelo Cliente-Servidor

- Variação do microkernel
- Realiza a distinção dos processos entre:
  - **Servidores**
    - Prestadores de serviços.
  - **Clientes**
    - Utilizadores de serviços.

Modelo Cliente-Servidor

- Comunicação é feita por troca de mensagens
  - Cliente que quer serviço monta mensagem e envia para o respectivo servidor
  - Servidor realiza o trabalho e envia mensagem-resposta
- Não há obrigatoriedade de clientes e servidores estarem na mesma máquina
- Clientes não precisam saber a localidade da entrega de suas mensagens

Modelo Cliente-Servidor



Máquinas Virtuais

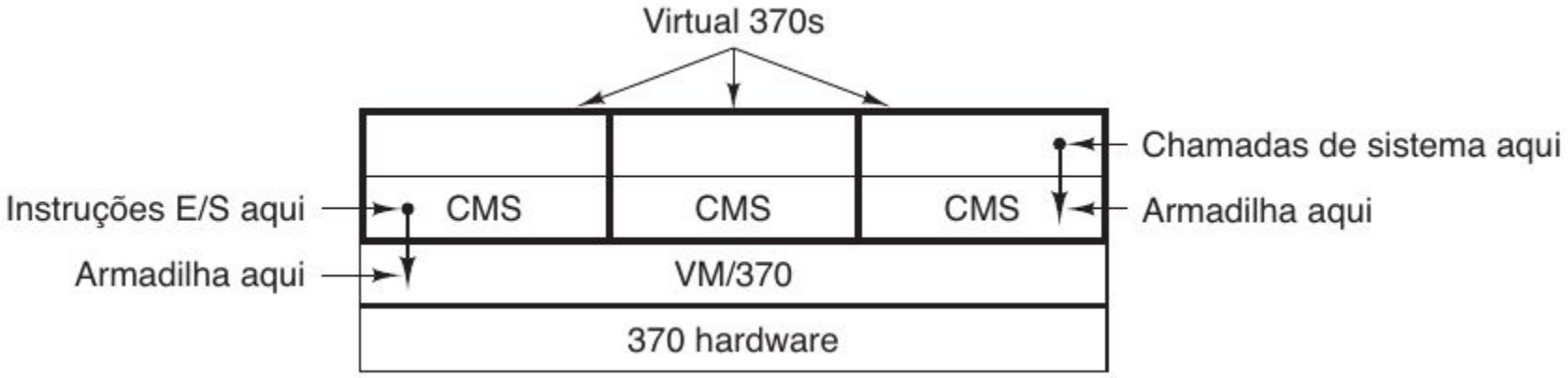
- Na época dos sistemas batch (em lote), OS/360, **usuários queriam trabalhar interativamente em um terminal**
- Quando surgiram sistemas de compartilhamento de tempo, estes eram grandes e lentos, e, por consequência, foram abandonados
- Do outro lado, um outro grupo desenvolveu uma versão diferente de tecnologia chamada z/VM

Máquinas Virtuais

- Por trás da tecnologia do z/VM, estava um princípio muito simples:
  - **Um sistema de compartilhamento de tempo fornece:**
    - **Multiprogramação**
    - **Máquina estendida**
- Dessa forma o VM/370 (a partir do z/VM) separou completamente essas duas funções

Máquinas Virtuais

- O núcleo do sistema, que foi chamado de **monitor de máquina virtual**, operava direto no hardware com multiprogramação, fornecendo não uma, mas várias máquinas virtuais para a outra camada.



Máquinas Virtuais

- Essas **máquinas virtuais**, ao invés de máquinas estendidas, **são cópias exatas do hardware**
  - Modos, E/S e tudo mais
- Por serem cópias, cada máquina virtual poderia executar qualquer SO sobre hardware

Máquinas Virtuais

- Quando um programa no SO do usuário executa uma chamada de sistema:
  - O **desvio** é realizado para a própria **máquina virtual**
  - O SO da máquina virtual emite instruções normais de hardware
  - Essas **instruções são desviadas para o VM/370** que realiza a execução real no hardware
  - O resultado é devolvido para o SO da máquina virtual, pelo VM/370

Máquinas Virtuais - Redescobrimento

- À época, a ideia de virtualização foi um pouco ignorada
  - Falta de visão e oportunidade
  - Com o tempo, novas necessidades e tecnologias resgataram o interesse nessa estrutura
- Necessidades:
  - Empresas tradicionais tinham seus próprios servidores
  - Virtualização permitia ter vários servidores de maneira isolada, em uma mesma máquina física

Máquinas Virtuais - Redescobrimento

- Oportunidade e tecnologia:
  - Crescimento de uso de hospedagem de páginas web
  - Antes, clientes precisavam escolher entre:
    - Hospedagem Compartilhada:
      - Sem qualquer controle no servidor
    - Hospedagem Dedicada:
      - O cliente tem uma máquina para ele, com total controle

Máquinas Virtuais - Redescobrimento

- Oportunidade e tecnologia:
  - Com a virtualização, uma empresa consegue prover um sistema com controle total para o cliente, que funciona por trás de uma máquina virtual
  - Usuário tem a impressão de ter uma máquina só para ele

Máquinas Virtuais

- Particularidades de Máquinas Virtuais:
  - Para executar um software de máquina virtual, a **CPU precisa ser virtualizável (1/2)**
    - Necessidade de instruções privilegiadas serem desviadas para um **monitor de máquina virtual**
    - Com o desvio, a instrução é emulada em software

Máquinas Virtuais

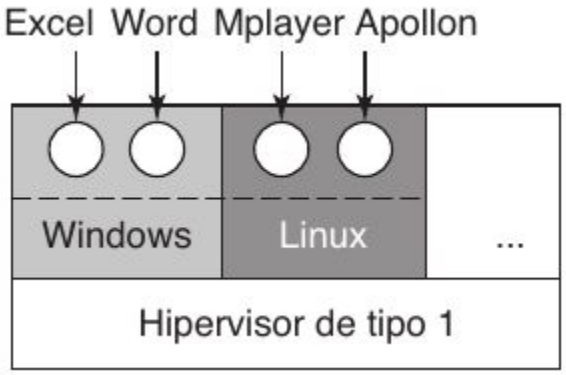
- Particularidades de Máquinas Virtuais:
  - Para executar um software de máquina virtual, a **CPU precisa ser virtualizável (2/2)**
    - Sem essa capacidade, algumas CPU tem o comportamento de ignorar instruções privilegiadas, se utilizadas em modo usuário
    - Houveram emuladores para sanar, porém muito lentos

Máquinas Virtuais

- Com o tempo, projetos de pesquisa levaram a produtos famosos no campo da virtualização:
  - VMware Workstation, Xen
  - KVM (linux), VirtualBox (Oracle) e Hyper-V (Microsoft).
- Nesses produtos e tecnologias anteriores, três concepções de Máquinas Virtuais podem ser apontadas:
  - Hipervisor tipo 1, tipo 2 e tipo 2 prático

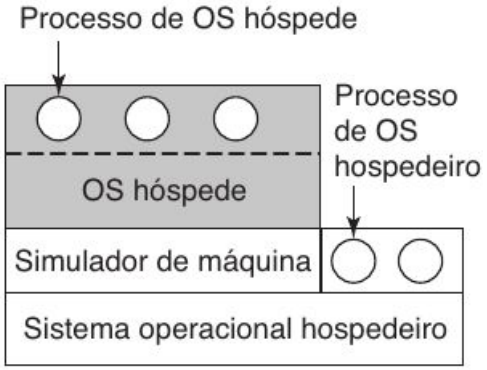
Máquinas Virtuais - Hipervisor Tipo 1

- Contém o monitor de máquina virtual.
- É a execução básica de um gerenciador para diferentes Sistemas Operacionais.
- Hipervisor tem contato direto com o hardware



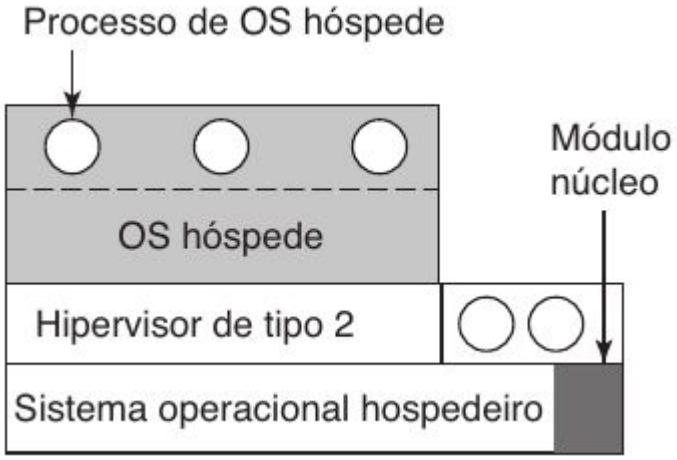
Máquinas Virtuais - Hipervisor Tipo 2 Puro

- Objetivo de melhorar a performance do Tipo 1
- Utiliza *cache* de blocos de código para acelerar
  - Conhecido como **simuladores de máquinas**
- Ainda não eram bons os suficiente para propósitos comerciais



Máquinas Virtuais - Hipervisor Tipo 2 Prático

- Acréscimo de modo núcleo ligando o hipervisor ao hardware
- Melhoria significativa de desempenho
- É o conceito mais empregado, atualmente

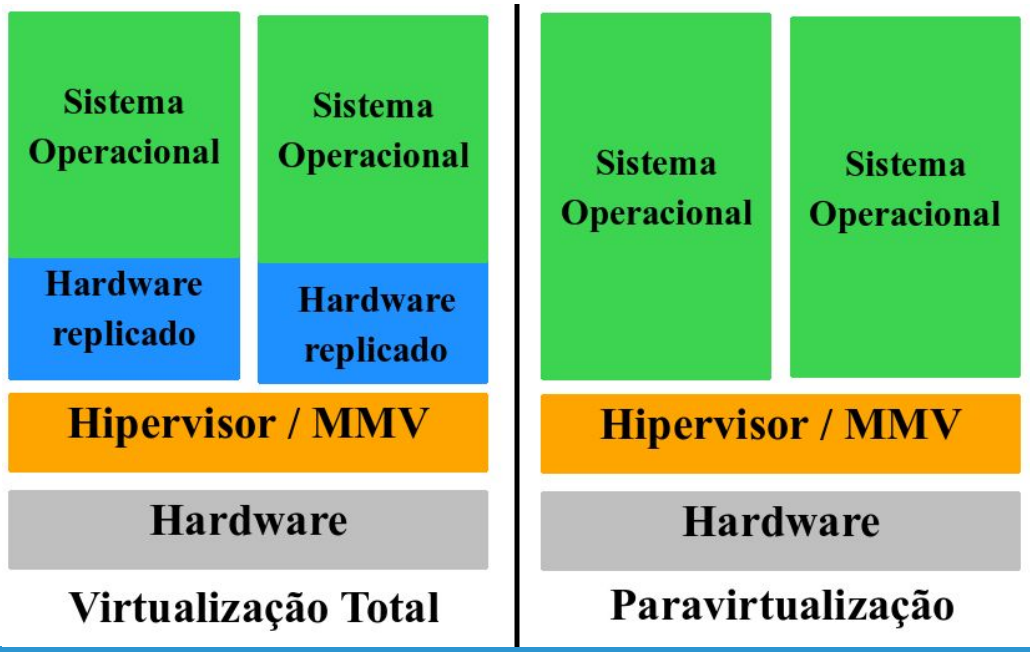


Máquinas Virtuais

- A diferença entre hipervisores tipo 1 e 2 reside na existência de um SO hospedeiro
  - Esse SO hospedeiro possui um sistema de arquivos
  - Tipo 1 tem que fazer tudo por ele mesmo
  - Tipo 2 instala o SO hóspede em um disco virtual
    - Um arquivo gigante no sistema de arquivos do SO hospedeiro

Máquinas Virtuais

- Além disso, outra abordagem para gerenciamento de instruções de controle, seria modificar o SO hóspede para comunicar-se diretamente com o Hipervisor, chamada de **Paravirtualização**



Máquinas Virtuais - Máquina Virtual Java (JVM)

- Outra maneira de utilização de máquinas virtuais
  - Execução de programas
- Programa escrito em java é compilado para a JVM e executado por um programa interpretador da JVM
- Dá portabilidade a programas escritos em java

Exokernels

- Exokernels ou exonúcleos utilizam o conceito de multiplexação segura de hardware
- Em vez de clonar o hardware, a estratégia é dividir os recursos
- Dessa forma, o hardware está visível às aplicações
- Cada usuário possui a sua fração do recurso para utilizar

Exokernels

- Essa estrutura é dividida em duas partes:
  - Exokernel:
    - Faz a multiplexação segura entre recursos de hardware, protegendo os mesmos
  - LibOS (sistemas operacionais biblioteca):
    - Conjunto de bibliotecas para gerenciar recursos e oferecer abstrações de alto nível para as aplicações

Sistemas Operacionais e a Linguagem C

- Linguagem C:
  - Kernels são geralmente escritos em C
  - C possui ponteiros explícitos
    - Ponteiros são poderosos
    - São a fonte de erros de diversos programas
  - Armazenamento em C é estático ou explicitamente alocado
  - A confiabilidade do código está no programador
    - Não possui pacotes, classes, objetos, segurança ou coletor

```
char c1, c2,
*p;
c1 = 'k';
p = &c1;
c2 = *p;
```

Sistemas Operacionais e a Linguagem C

- Linguagem C:
  - Arquivos de cabeçalho.
    - Constantes, macros e compilação condicional

```
#ifdef __i386__
    #include <asm/xor_32.h>
#else
    #include <asm/xor_64.h>
#endif
```

Sistemas Operacionais e a Linguagem C

- Sistemas Operacionais são a combinação de diversos arquivos-objeto
  - Milhares de linhas de código
  - Fazem uso do programa make para gerenciar modificações e geração desses arquivos objeto (recompila só o que é necessário)
  - Uma vez ligados, o computador pode ser reinicializado
    - O que não foi incluído estaticamente, se-lo-á dinamicamente carregado.
      - Drivers, sistemas de arquivos, etc.

Conclusão

- SOs são grandes, inflexíveis, inconfiáveis, inseguros e cheio de erros
  - Consequentemente, muita coisa muda o tempo todo
  - Diversas pesquisas são feitas nessa área
- O domínio de SO auxilia na execução de diferentes tarefas computacionais
  - Domínio de recursos
  - Entendimento do ambiente

Conclusão